

Paralegal: Grounded Legal Memo RAG

Technical Report

Paralegal is a compact document-understanding and retrieval-augmented drafting system for legal and regulatory materials. It turns messy source files into inspectable evidence, uses that evidence to draft a cited legal memo, and records operator edits so later drafts can reflect recurring preferences.

The main design goal is simple: every useful statement in the draft should be traceable back to source material. When the system cannot support a fact, it should make that uncertainty visible instead of filling the gap with fluent but unsupported text.

1 Scope

The project focuses on one end-to-end workflow:

1. Ingest legal-style documents.
2. Extract text, layout-aware blocks, warnings, and structured fields.
3. Build a searchable evidence index.
4. Retrieve passages for a drafting task.
5. Generate a cited memo draft.
6. Validate that draft against the retrieved evidence.
7. Save operator edits and convert them into reusable drafting preferences.
8. Run a small evaluation suite that makes the behavior inspectable.

The chosen output is a cited legal memo draft with sections for parties and documents, timeline, key facts, document issues, open questions, and evidence used.

2 Coverage Matrix

Area	Where it is handled
Document processing	<code>paralegal.ingestion</code> extracts native text, routes OCR, records confidence and warnings, and writes normalized evidence blocks plus structured fields.
Retrieval and grounding	<code>paralegal.retrieval</code> builds sparse and vector indexes, fuses rankings, optionally reranks, and returns inspectable evidence ids.

Draft generation	<code>paralegal.generation</code> drafts a cited memo from retrieved evidence, sanitizes citations, validates grounding, and falls back safely when needed.
Improvement from edits	<code>paralegal.feedback</code> stores operator edits, classifies reusable changes, aggregates learned patterns, and feeds them into later drafts.
Code quality and design	The code is split into small modules for ingestion, retrieval, generation, feedback, providers, evaluation, and UI. Public functions and data models have docstrings.
Documentation and clarity	<code>README.md</code> , <code>docs/developer-guide.md</code> , this report, sample outputs, screenshots, and tests document how to run and inspect the system.

3 Document Processing

The ingestion layer accepts PDFs, text, markdown, PNG, JPG, and JPEG files. Clean PDFs are handled with native PDF text extraction first because that path is faster, cheaper, and often more accurate than OCR. Scanned or image-heavy files can route through OCR.

The implemented OCR options are:

- Mistral OCR when `MISTRAL_API_KEY` is configured.
- Local Docling OCR/layout parsing through `-ocr-backend docling`.
- A safe low-confidence fallback record when image OCR is not available.

Each extracted block is normalized into a `DocumentBlock` with:

- `doc_id`
- `source_file`
- `page`
- `block_index`
- `text`
- `confidence`
- `warnings`
- `extraction_method`
- a stable citation id such as `DOC2:p6:b1`

That block format is the core contract between ingestion, retrieval, generation, UI evidence inspection, and evaluation. The system also writes document summaries with structured fields for parties, dates, addresses, amounts, signatures, deadlines, and unclear spans.

The processing layer is deliberately conservative. If OCR fails, if a page has no native text, or if text quality looks poor, the warning follows the block into downstream outputs. That makes extraction risk visible to the draft generator and to the operator.

4 Retrieval and Grounding

Retrieval is local and inspectable. The index uses SQLite FTS5 for sparse keyword matching, a vector store for semantic search, reciprocal-rank fusion to combine both rankings, and an optional local cross-encoder reranker.

The default retrieval flow is:

1. Expand the query with small legal-domain synonym sets.
2. Search SQLite FTS5 with BM25-style ranking.
3. Search the vector index with the configured embedding provider.
4. Combine sparse and dense results with reciprocal-rank fusion.
5. Optionally rerank top candidates with `BAAI/bge-reranker-base`.
6. Return evidence chunks with exact citation ids and source metadata.

Embedding options are:

- OpenAI embeddings when configured.
- Local Qwen/Qwen3-Embedding-0.6B through Sentence Transformers.
- Deterministic hashing embeddings for no-key tests and offline baseline runs.

The important product behavior is not just finding relevant text. The system returns an evidence bundle that can be inspected in the UI and written to `sample_outputs/evidence_map.json`. Generated citations are then checked against that bundle.

5 Draft Generation

Generation is evidence-first. The generator retrieves evidence, adds useful structured-field support chunks, loads any learned edit preferences, and then drafts the memo through one of three paths:

- OpenAI, when explicitly enabled and configured.
- Local Ollama, when `-use-ollama` is set.
- A deterministic offline generator when no model is available or a model result fails grounding checks.

The model prompt tells the generator to use only the provided evidence ids. After generation, the system sanitizes common citation mistakes, removes weakly supported lines, and validates the final memo.

Validation checks:

- every citation id appears in the retrieved evidence bundle,
- factual bullets carry citations,
- required memo sections are present,
- the draft does not look truncated,

- cited factual lines have enough lexical overlap with their cited evidence.

This is intentionally stricter than a typical demo prompt. The draft should degrade into open questions or a deterministic fallback before it presents unsupported claims as facts.

6 Improvement from Operator Edits

The edit loop is designed to learn reusable drafting behavior, not just store two versions of a memo.

When an operator saves an edit, the system stores:

- the original draft,
- the edited draft,
- the visible evidence ids,
- the task type,
- a timestamp,
- a structured edit analysis.

If OpenAI is available, the edit classifier asks for structured reusable patterns. Otherwise, a heuristic classifier looks for common signals such as removed uncited claims, added cited facts, stronger citation density, tighter prose, or new open questions.

The aggregate file `data/feedback/learned_patterns.json` keeps a compact list of learned patterns and recent before/after examples. Future drafts load that summary as prompt context. In the current sample run, the simulated edit teaches the system to carry operator-added cited facts into future drafts when similar evidence appears.

7 UI

The Streamlit UI exposes the workflow in one place:

- choose local or API-backed providers,
- download public samples or upload documents,
- ingest documents,
- build the evidence index,
- generate a memo,
- inspect retrieved evidence,
- edit the memo,
- save feedback,
- run evaluation.

The UI is not meant to be a production frontend. It is a fast way to inspect the pipeline, exercise the grounding loop, and demonstrate how evidence connects to the draft.

8 Local and API Deployment Modes

The project supports two practical modes.

8.1 Local-first mode

The WSL local stack uses Docling, Qwen embeddings, BGE reranking, and Ollama drafting. This keeps documents on the machine by default. It is the better direction for confidential legal work when hardware is available.

Local CPU inference is slower, especially for OCR and drafting. The WSL runner detects whether PyTorch can safely use CUDA. Unsupported visible GPUs fall back to CPU instead of failing midway through the run.

8.2 API-backed mode

The API-backed path can use Mistral OCR, OpenAI embeddings, OpenAI structured extraction, and OpenAI drafting. This improves quality and latency on weaker hardware, but it should be used only when the document handling policy allows external processing.

The provider choice is explicit. API providers are optional and controlled by environment variables and CLI flags.

9 Sample Inputs and Outputs

The public sample workflow uses four legal or regulatory inputs:

File	Purpose
<code>ftc_chegg_complaint.pdf</code>	Regulatory complaint with factual allegations and recurring-charge language.
<code>ftc_chegg_stipulated_order.pdf</code>	Order document with monetary and injunctive relief language.
<code>doj_schnitzer_consent_decree.pdf</code>	Consent decree with compliance obligations and notice provisions.
<code>sec_scanned_lease_page_001.jpg</code>	Image-only lease exhibit page to exercise OCR-required handling.

Representative outputs are checked into `sample_outputs/`:

- `generated_memo.md`
- `evidence_map.json`
- `evaluation_report.md`
- `evaluation_report.json`

The generated memo includes citation ids on factual bullets. The evidence map preserves the retrieved source chunks so a user can trace each citation back to the underlying text.

10 Evaluation Approach

The evaluation suite is intentionally small and transparent. It is designed to show whether the system is behaving correctly, not to claim broad legal accuracy.

It measures retrieval term recall at 5, source recall at 5, mean reciprocal rank, extracted document and block counts, structured field counts, extraction warnings, citation validity, uncited factual lines, weakly supported cited lines, and edit-learning persistence.

Area	Result
Documents processed	4
Evidence blocks indexed	141
Mean term recall@5	0.67
Mean source recall@5	1.00
Mean reciprocal rank	0.88
Extracted parties	6
Extracted dates	19
Extracted addresses	9
Extracted amounts	11
Extracted signatures	9
Extracted deadlines	6
Extraction warnings	11
Citation validation	Passed
Invalid citations	0
Uncited factual lines	0
Weakly supported cited lines	0
Stored feedback records	1

The weakest retrieval case is the image-only SEC lease page. Source recall still succeeds, but term recall is lower because OCR quality and scanned-page structure affect which exact terms appear in the extracted chunks. That is the expected failure mode, and it is why extraction warnings and low-confidence records remain visible.

11 Assumptions

- Public documents are acceptable for demonstration because they can be inspected without exposing confidential client material.
- Synthetic documents are useful for tests, but public inputs are better for showing real extraction and retrieval behavior.
- Legal correctness is outside the system boundary. The system is evaluated on grounding, traceability, workflow design, and reliability.
- A human operator remains responsible for legal judgment and final edits.

- Fine-tuning is not used. The edit loop uses prompt memory because the amount of feedback data is too small for meaningful model training.
- The Streamlit UI is a demo surface, not an access-controlled production application.

12 Design Choices

The implementation favors inspectability over model complexity. SQLite, JSONL, Markdown outputs, and simple provider adapters make it easy to see what happened at each stage. That matters more here than hiding the workflow behind a larger service framework.

The retrieval stack uses hybrid search because legal documents often need exact term matching and semantic recall at the same time. BM25 catches names, dates, statutes, and monetary values. Embeddings help when the query is phrased differently from the source. Reranking is optional because it adds latency and local model dependencies.

The deterministic fallback generator is not as polished as a strong LLM, but it is valuable. It means the pipeline can still produce a grounded, testable output without network access or model availability.

OCR is the biggest quality variable. Local OCR is better for privacy. API OCR is better for quality and speed on weak hardware. The system supports both because the right answer depends on the deployment environment and document sensitivity.

13 Production Path

A production version should keep the same core contracts but harden the surrounding system:

- replace Streamlit with an authenticated web application,
- add matter-level permissions and audit logging,
- encrypt stored documents and derived artifacts,
- add retention controls and deletion workflows,
- store evidence, feedback, and evaluations in a database,
- run OCR and embeddings as background jobs,
- add queueing, retries, and observability,
- support document-level access controls during retrieval,
- add stronger claim-level attribution checks,
- expand evaluation sets with real labeled matters,
- require human approval before any draft leaves the workspace.

The current repo is a reference implementation of the workflow. The strongest parts to carry forward are the evidence-id contract, hybrid retrieval, citation validation, and edit-learning loop.

14 How to Reproduce

Run the baseline:

```
venv\Scripts\paralegal.exe all
```

Run the tests:

```
venv\Scripts\python.exe -m pytest
```

Launch the UI:

```
venv\Scripts\streamlit.exe run app.py
```

Run the local WSL stack:

```
bash scripts/run_wsl_local.sh
```